

FPGA Implementations of the Round Two SHA-3 Candidates

Brian Baldwin*, Andrew Byrne†, Liang Lu‡, Mark Hamilton*, Neil Hanley*, Maire O'Neill‡ and William P. Marnane*

* Claude Shannon Institute for Discrete Mathematics, Coding and Cryptography.

Department of Electrical & Electronic Engineering, University College Cork, Cork, IRELAND

Email: {brianb, markh, neilh, liam}@eleceng.ucc.ie

† School of Mathematical & Geospatial Sciences, RMIT University, Melbourne, Australia

Email: andrew.byrne@rmit.edu.au

‡ Institute of Electronics, Communications and Information Technology, Queens University Belfast, Belfast, UK

Email: {l.lu, m.oneill}@ecit.qub.ac.uk

Abstract—The second round of the NIST-run public competition is underway to find a new hash algorithm(s) for inclusion in the NIST Secure Hash Standard (SHA-3). This paper presents the full implementations of all of the second round candidates in hardware with all of their variants. In order to determine their computational efficiency, an important aspect in NIST's round two evaluation criteria, this paper gives an area/speed comparison of each design both with and without a hardware interface, thereby giving an overall impression of their performance in resource constrained and resource abundant environments. The implementation results are provided for a Virtex-5 FPGA device. The efficiency of the architectures for the hash functions are compared in terms of throughput per unit area.

To the best of the authors' knowledge, this is the first work to date to present hardware designs which test for all message digest sizes (224, 256, 384, 512), and also the only work to include the padding as part of the hardware for the SHA-3 hash functions.

I. INTRODUCTION

NIST is currently holding a hash competition [1] to select a new hash algorithm(s), called SHA-3 (or the Advanced Hash Standard (AHS)), for the purpose of ultimately superseding the functions in the SHA-2 family. The contest initially received 64 submissions from designers worldwide, with 51 of these designs progressing through to round one of the contest. The round two candidates were announced in Q3 of 2009 and the competing designs were reduced to 14. Round three is to be announced in Q3 of 2010, with the winner expected to be announced in 2012.

These hash algorithms are available for public comment and scrutiny and NIST has stated that computational efficiency of the algorithms in hardware, over a wide range of platforms, will be addressed during the second round of the contest [1]. Some initial work has already been completed on documenting the hardware speed and size for different types of hardware [2], [3]. The work in this paper is a continuation of work in [4], in which three of the five selected hash functions progressed to round 2, CubeHash, Grøstl and Shabal. We also present updated round two results for these designs.

The rest of this paper is organised as follows. In section II we give a brief description of our design decisions and our testing methodology. Section III gives an overview of hash

functions and the hash function architectures, with subsections III-C–III-J respectively describing the FPGA implementations of the hash functions in this case study. For each hash function, its specification is briefly described; an exploration of the design space is presented; and implementation results on the Virtex-5 FPGA platform are supplied. Section IV gives results for the designs, and Section V concludes.

II. FAIR COMPARISON METHODOLOGY

In the NIST competition specifications [1], 6.C, *Round 2 Technical Evaluation* gives the criteria for hardware testing; "Round 2 testing by NIST will be performed on the required message digest sizes" and "the calculation of the time required to compute message digests for various length messages". As such, the authors felt that for a complete analysis of the hash functions as required by NIST, it was necessary to implement as many designs as was necessary for full coverage of all of the message digest sizes, {224,256,384,512}. While in some cases, all four variants can be output from a single design, where only the initial vectors (IV) and truncation differ, others require two or even three different designs to produce the four output digests. The current hash standard, SHA-2, was also implemented as a reference point. To allow fair comparison between different designs, all designs were implemented in slice logic, using distributed memory instead of dedicated block memory where required. Although this does not make the best use of the FPGA, all the designs and their variants can thus be fairly measured and analysed.

For the calculation time, it was decided to also include the padding stages in hardware, required both for the purpose of testing the hash functions against the Known Answer Tests (KAT) values provided by each designs submission package, and also to give a fair and accurate timing report inclusive of all stages required to hash a message. As there are a number of different padding schemes, each of which have differing number of rounds required to complete, the padding also has a bearing on the calculation time.

We developed a hardware wrapper interface [5] which necessarily needed to be published separately due to space lim-

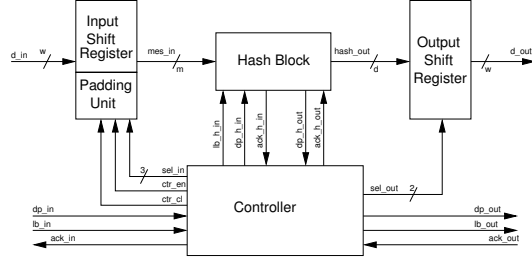


Fig. 1. Wrapper Interface

itations. This wrapper, made freely available on our website to allow others to test their designs, <http://www.ucc.ie/en/crypto/>, shown in Fig. 1, produces the padding scheme required for a particular hash function as well as setting the input and output bus to a particular size. It allows re-use of any padding scheme that can be used in multiple hash functions. In this case we set the i/o bus widths, w , to 32-bits, a standard word size. This models a realistic communications system, and shows any bandwidth limitations, as any hash function requiring a large message size, m , will be subject to a latency of m/w clock cycles. Our hash functions necessarily take this latency into account, and we attempted to optimise our designs so as to minimise as much as possible any delays due to this loading of data, i.e. the hash function iteration should if possible take longer than the time required to load a message. We showed in [5] that, while the wrapper itself does not affect the clock frequency, counters in the padding block may form the critical path and thus affect the timing. As such, in the interest of fairness we present area and frequency results both inclusive and exclusive of the wrapper.

III. OVERVIEW OF THE HASH FUNCTION ARCHITECTURES

Table I lists the constructions of the different SHA-3 hash functions and their variants as well as the various inputs and state sizes in bits. As the {224} variant is almost identical to the {256} and similarly, the {384} to the {512} we omit these values. The only notable difference being Keccak, where the message size increases to 1152 for {224} and 832 for {384}, and Luffa, where the state size decreases to 1024 for {384}. "Structure" defines the hash function type, for example, in HAIFA (Hash Iterative Framework) [6] based designs, the counter is fed in with the message, whereas, for Merkle-Damgård [7] [8], it is not. Table II lists the different padding schemes used by the hash functions.

Understandably this review is somewhat brief and so for a full description of each algorithm, the interested reader is referred to the SHA-3 submission documentation [9].

In the design of the hash function architectures described in this paper, the main goal was to give a baseline comparison between the hash functions using area and throughput. We calculate the throughput as follows:

$$\text{Throughput} = \frac{\# \text{ Bits in a message block} \times \text{Maximum clock frequency}}{\# \text{ Clock cycles per message block}}$$

The FPGA platform targeted in the study was the Xilinx Virtex-5 *xc5v1x330T-2-ff1738*. Each hash function design was

TABLE I
HASH FUNCTION INTERNALS

Design	Structure	Type	224/256				384/512			
			Counter	Message	Salt	State	Counter	Message	Salt	State
SHA-2	Merkle-Damgård	Add-XOR-Rotate	64	512	0	512	128	1024	0	1024
Blake	HAIFA	Add-XOR-Rotate	64	512	128	512	128	1024	256	1024
BMW	Iterative	Add-XOR-Rotate	64	512	0	2048	64	1024	0	4096
Cubehash	Iterative	Add-XOR-Rotate	0	256	0	1024	0	256	0	1024
Echo	HAIFA	AES based	64	1536	128	2048	64	1536	128	2048
Fugue	Iterative	AES based	64	32	0	960	64	32	0	1148
Grøstl	Iterative	AES based	64	512	0	512	64	1024	0	1024
Hamsi	Conc-Permute	Serpent based	64	32	0	512	64	64	0	1024
JH	Iterative	Block Cipher based	128	512	0	1024	128	512	0	1024
Keccak	Sponge	Add-XOR-Rotate	0	1088	0	1600	0	576	0	1600
Luffa	Sponge	S-box based	0	256	0	768	0	256	0	1280
Shabal	Iterative	Add-XOR-Rotate	0	512	0	1408	0	512	0	1408
SHAvite-3	HAIFA	AES based	64	512	256	256	128	1024	512	512
SIMD	Iterative	Block Cipher based	64	512	0	512	64	1024	0	1024
Skein	UBI	Add-XOR-Rotate	96	512	0	512	96	512	0	512

TABLE II
PADDING SCHEMES

Design	Padding Scheme
SHA224/256	1, 0's until congruent (448 mod 512), 64-bit message length
SHA384/512	1, 0's until congruent (896 mod 1024), 128-bit message length
Blake224	1, 0's, until congruent (448 mod 512), 64-bit message length
Blake256	1, 0's, until congruent (447 mod 512), 1, 64-bit message length
Blake384	1, 0's, until congruent (895 mod 1024), 128-bit message length
Blake512	1, 0's, until congruent (894 mod 1024), 1, 128-bit message length
BMW224/256	1, 0's until congruent (448 mod 512), 64-bit message length
BMW384/512	1, 0's until congruent (960 mod 1024), 64-bit message length
Cubehash	1, 0's until a multiple of 256 (256 = 8 * b, b=32)
Echo224/256	1, 0's until congruent (1392 mod 1536), 16-bit message digest, 128-bit message length
Echo384/512	1, 0's until congruent (880 mod 1024), 16-bit message digest, 128-bit message length
Fugue	0's until a multiple of 32, 64-bit message length
Grøstl224/256	1, 0's until congruent (448 mod 512), 64-bit block counter
Grøstl384/512	1, 0's until congruent (960 mod 1024), 64-bit block counter
Hamsi224/256	1, 0's until a multiple of 32, 64-bit message length
Hamsi384/512	1, 0's until a multiple of 64, 64-bit message length
JH	1, 0's until congruent (384 mod 512), 128-bit message length, min 512-bits added
Keccak224	1, 0's until a multiple of 8, append 8-bit representation of 28, append 8-bit representation of 1152/8, 1, 0's until a multiple of 1152
Keccak256	1, 0's until a multiple of 8, append 8-bit representation of 32, append 8-bit representation of 1088/8, 1, 0's until a multiple of 1088
Keccak384	1, 0's until a multiple of 8, append 8-bit representation of 48, append 8-bit representation of 832/8, 1, 0's until a multiple of 832
Keccak512	1, 0's until a multiple of 8, append 8-bit representation of 64, append 8-bit representation of 576/8, 1, 0's until a multiple of 576
Luffa	1, 0's until a multiple of 256
Shabal	1, 0's until a multiple of 512
SHAvite3-224/256	1, 0's until congruent (432 mod 512), 64-bit message length, 16-bit digest length
SHAvite3-384/512	1, 0's until congruent (880 mod 1024), 128-bit message length, 16-bit digest length
Simd224/256	0's until a multiple of 512, extra block with message length
Simd384/512	0's until a multiple of 1024, extra block with message length
Skein	0's if multiple of 8, else 1, 0s, until a multiple of 512

implemented using VHDL, and Synthesis, Place and Route were carried out using Xilinx ISE v9.2i. We measure area of our hash function designs in FPGA slices, as given by the Map report.

A. BLAKE - Aumasson et al.

All BLAKE hash functions use the following steps to compress a message:

- The inner state, v is initialized from the initial vectors (IV), a salt, and a counter which counts the message.
- Once the state v is initialized, it is subdivided into 16 blocks and the compression function iterates a series of 10 rounds for BLAKE-28/32 and 14 rounds for BLAKE-48/64.
- Each round comprises a series of additions, xors and rotations between four of the blocks along with a block of the initial message and an initial vector (IV).
- The blocks are called in a column-step and diagonal-step method, thus allowing four blocks to be operated on in

parallel.

- A final stage xors the initial chain and the salt with the blocks to produce the hashed message.

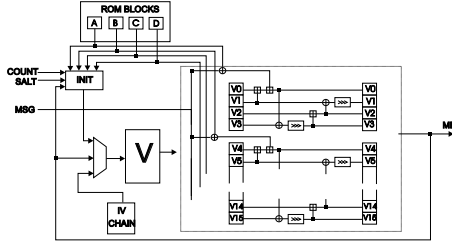


Fig. 2. Blake

For our implementation of BLAKE we further subdivide the compression function into two identical sections, to allow reuse of the component blocks and thereby reducing the area. This subdivision increases the latency of the hash permutation to complete a round from two to four clock cycles, but reduces the critical path from four adders to two adders thus increasing the maximum frequency of the permutation. For the larger variant which requires 32 clocks to load a 1024 bit message, it ensures there is no delay where the hash function needs to wait for loading to complete. Fig. 2 shows the modified design. The Adders and XORs were generated in a generic fashion (using the '+' operator of the IEEE.std logic unsigned package) and the rotation operations were implemented through simple wiring, with multiplexers to select the particular subround rotation. The 16 constants required by the initialisation and round stages are stored in distributed ROM.

B. Blue Midnight Wish - Gligoroski et al.

BMW consists of three core functions:

- $f_0 : \{0, 1\}^{2m} \rightarrow \{0, 1\}^m$
- $f_1 : \{0, 1\}^{3m} \rightarrow \{0, 1\}^m$
- $f_2 : \{0, 1\}^{3m} \rightarrow \{0, 1\}^m$

where m is the number of bits in the message block $M^{(i)}$ and the double pipe value $H^{(i-1)}$. Each of $M^{(i)}$, $H^{(i)}$, $Q_a^{(i)}$ and $Q_b^{(i)}$ are represented as sixteen 32 or 64 bit blocks depending on the variant.

- f_0 takes $M^{(i)}$ and $H^{(i-1)}$ as its inputs and produces the first half of the quadrupled-pipe value $Q_a^{(i)}$. f_0 consists of 80 additions/subtractions. The function also consists of many XOR's, bitwise shifts and rotations.
- f_1 takes $M^{(i)}$, $H^{(i-1)}$ and $Q_a^{(i)}$ as its inputs and produces the second half of the quadrupled-pipe value $Q_b^{(i)}$. The quadrupled-pipe is then $Q^{(i)} = (Q_a^{(i)}, Q_b^{(i)})$. f_1 is the most complex of the functions performed by BMW. It consists of two sub functions ER_1 and ER_2 . Both ER_1 and ER_2 contain sixteen modulo 32 addition operations but ER_1 contains much more bitwise shift and rotate operations than ER_2 . Both functions use an *AddElement* operation that uses modulo 32 additions, subtractions and rotations to combine a block of the message and of the double-pipe with a predefined set of constants.

- The final function f_2 takes $M^{(i)}$, $Q_a^{(i)}$ and $Q_b^{(i)}$ as inputs and produces the new double-pipe value $H^{(i)}$. f_2 consists of XOR, bitwise shift, rotation and modulo 32 addition operations.

A pipelined design was chosen for implementation due to the large amount of additions that need to be performed. Each of the functions f_0 , f_1 and f_2 make up a stage in the pipelined design. One operation of the compression cycle therefore takes three clock cycles. A diagram of the pipelined design is shown in Figure 3.

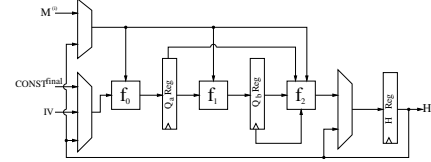


Fig. 3. BMW Hash Function

C. ECHO - Orange Labs

The compression function (CF^E) for ECHO, operates iteratively as follows: $V_i = CF^E(V_{i-1}, M_i, C_i, SALT)$ where V_{i-1} is the current value of the chaining variable, M_i is the current message block, C_i is a counter and $SALT$ is a sub-key. All ECHO hash functions use the following steps to compress a message.

- Initialisation: Counter C_0 is set to 0. Each word of V_0 (chaining variable) is given an initial value based on the message digest size. This is followed by the padding.
- Message Compression: The compression function (CF^E) is similar to the AES structure, which consists of the sequential operations of *SubWords* (an S-Box look-up table for each 128-bit word), *ShiftRows* (shifts the 128-bit words of the state, S) and *MixColumns* (four 128-bit columns of the state S are considered as 64 byte wide columns. The AES *MixColumns* operation can be applied to each of these columns). Two sub-keys for CF_{512}^E and CF_{1024}^E will be generated based on the value of the counter and the $SALT$.
- Final Compression: After 8 iterations of CF_{512}^E (10 for CF_{1024}^E), a final compression function CF^{Efinal} is executed.

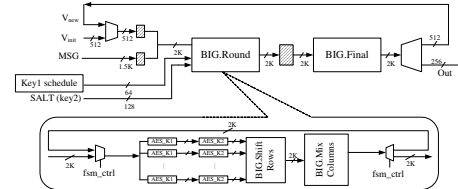


Fig. 4. Echo High Performance Architecture

In this paper, we designed and evaluated FPGA implementations of the ECHO hash function with an output of $\{256 \& 512\}$ bits. In the proposed design, an iterative architecture is

used with one *BIG.ROUND* function and a finite state machine (FSM) to control the data-path. Within the *BIG.ROUND* function, 16 pairs of the AES functions are processed in parallel to improve the throughput rate. The S-boxes are implemented using distributed ROM memory. Sub-keys are pre-calculated prior to the compression function. An outline of this architecture is provided in Fig. 4. In this design, the *BIG.ROUND* operation is performed in one clock cycle.

D. Fugue - Halevi et al.

The Fugue hash function was designed by researchers at IBM. The following steps are used to compute the hash:

- All three variations take in an input stream of $4m$ bytes, where $m \geq 0$, along with an initial vector IV (28, 32, 48 and 64-bytes respectively for Fugue 224/256, 384 and 512).
- The input block of $4m$ bytes is parsed as m 4-byte words and passed one word at a time into the round transformation.
- The round transformation consists of an operation on an internal state defined as a matrix of n 4-byte columns (where n is 30, 48 and 36 for each of the variants).
- XOR operations and a number of rotation operations, dependent on the round number.
- After all the inputs have been processed, the state undergoes a final round transformation.
- Columns of the state are used as the output of the hash function.

The rotation blocks (ROR) operate on different sizes for each of the variants, but the overall design remains the same. Fig. 5 shows the operation of Fugue 256.

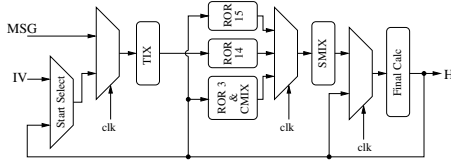


Fig. 5. Fugue 256

The initial round comprises a TIX stage (XOR, truncate, insert and XOR of bytes), a rotation by 3 bytes (ROR3) and a column mix (CMIX), all on each of the n blocks, followed by a super mix (SMIX) transformation. This transformation takes a 4×4 matrix of bytes and passes each byte through an S-box, followed by a linear transformation to generate diffusion. This linear transformation is similar to that employed in AES, however, unlike AES, there is cross-mixing between the columns. These steps are looped a number of times r depending on the variant, with $r = 2, 3, 4 \in \{224, 256, 384, 512\}$ per message block. The final round comprises of another two loops of a rotation by three bytes (ROR3), CMIX and SMIX (repeated p times), an XOR, and finally a rotation by 15 or 14 followed by an SMIX (repeated q times). For the different variants $p = 5, 18, 32 \in \{224, 256, 384, 512\}$ and $q = 13 \in \{224, 256, 384, 512\}$

In all variants, the CMIX and linear transformation operations are implemented using combination logic, and the rotation operation was implemented through rewiring. As defined in the specifications, for each design we declared four blocks where the state words are operated on and shifted along chains of 8-bits. The S-boxes are implemented using distributed ROM memory.

E. Hamsi - Küçük

The Hamsi hash function has a concatenate-permute-truncate construction and consists of four main operations:

- Message Expansion: The input message is expanded using linear codes. For Hamsi-224/Hamsi-256 the 32-bit input message is expanded to 256-bits, for Hamsi-384/Hamsi-512 the 64-bit input message is expanded to 512-bits.
- Concatenation: The expanded message is concatenated with an initial value or the output from the previous stage of the hash function.
- Non linear permutation P : The non linear permutation P is made up of three sub operations. The first operation XOR's the state with a table of predefined constants and also a counter. The second operation is the application of one of the Serpent [10] S-boxes. The final operation in P is a diffusion operation L . This operation consists of several bitwise shifts and XOR's.
- Truncation: The truncation stage reduces the Hamsi state down to the size of the input message. In the case of Hamsi-224 and Hamsi-384 the state must be further truncated to achieve message digest lengths of 224 and 384 bits, this is only done after the last message block has been processed.

A fully parallel design was chosen for implementation as shown in Figure 6. The non-linear permutation P was unrolled three times, more unrolling resulted in a congested design for the VHDL. Therefore it takes one clock cycle for a normal message block to be hashed and two clock cycles for the final message block.

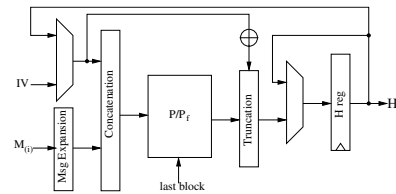


Fig. 6. Hamsi Hash Function

F. JH - Hongjun Wu

JH uses the same design for all four variants and is based on simple components. The compression function combines a 1024-bit previous hash block (H_{i-1}), a 512-bit message block (M_i) to produce a 1024-bit hash block (H_i).

The three main stages of the hash function are as follows:

- Initialisation: The initial hash value, H_0 is set according to the message digest size. This is followed by the padding.
- Compression: The compression function (CF^{JH}) is applied to each message block, M_i . This requires an initial xor between the lower 512-bits of H_{i-1} and M_i followed by a bijective function and an xor between the upper 512-bits of the updated H_{i-1} and M_i .
- Truncation: The message digest is determined by a truncation of the final hash value according to the message digest size.

The bijective function consists of 35 rounds, each consisting of an S-box, linear transformation and permutation, and a single final round consisting of just the S-box. Two 4-bit S-boxes are used, the selected table depending on the value of a round constant. It can also be viewed as a 5-bit to 4-bit substitution. The linear transformation implements a (4, 2, 3) maximum distance separable (MDS) code over $GF(2^4)$, and the permutation shuffles the output according to three distinct smaller permutations. The 256-bit round constants can be generated either in parallel with the data path or pre-computed and stored in memory where they can be re-used.

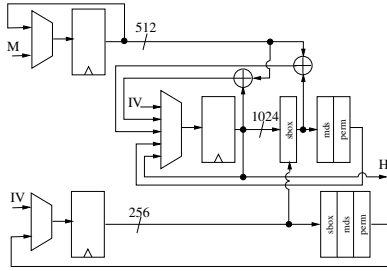


Fig. 7. JH Parallel Compression Function

In the design presented, the full 1024-bit data state is operated on at once. Each round completes in a single clock cycle. The 256-bit subkey state is calculated in parallel. This parallel architecture is illustrated in Fig. 7. The S-box and linear transformation functions are implemented as combinational logic, and the grouping and permutation functions are rewiring circuits. In the round constant data path, only the S-box corresponding to select bit '0' is required. Three registers are required for data storage, one each for the round constant, message block and data block respectively.

G. Keccak - Bertoni et al.

Keccak is a hash function based on the sponge construction [11]. The NIST submissions use the same KECCAK-f permutation for all variants, with different capacity (c), bitrate (r) and diversifier (d) values, where smaller digest sizes have a greater bitrate.

The hash function operation can be outlined as follows:

- Padding and Permutation: After padding, the KECCAK-1600 permutation consists of 24 rounds of five steps. Each round is identical except for the round constants.

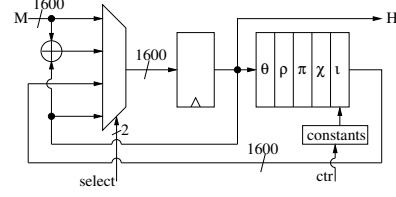


Fig. 8. Keccak-f(1600) Permutation

- Truncation: The message digest is determined by a truncation of the final hash value according to the message digest size.

The five steps of the permutation consist of addition and multiplication operations in $GF(2)$. The full round computes in a single clock cycle, and an extra clock is required for loading in of the message. The padded message of length r is loaded in by XOR'ing it with r bits of the state. The 64-bit round constants are defined as the output of a linear feedback shift register and can be pre-computed or generated as required. In the design presented, they are pre-computed and stored in distributed ROM. Only one register is required and is used to store the state value. The HDL implementation provided in the submission documentation was used as a reference for the permutation steps in our design.

H. Luffa - De Cannière et al.

The input message is split into 256-bit blocks arranged as 8 32-bit words, where each word consists of 4 bytes. Luffa-n uses the following operations to compute the hash of a message M :

- Initialisation. An initial set of hash values, V , are set to the corresponding IV. This is followed by padding and parsing.
- Message Compression. Each message block M_i is combined with the previous hash value H_{i-1} . During the round function, a message injection and permutation function are applied to these inputs. This process is illustrated in Fig. 9.
- Final Compression and Truncation. A finalization round is performed at the end with a blank M_i as input. This is the same as the round function but includes an extra XOR step on the output.

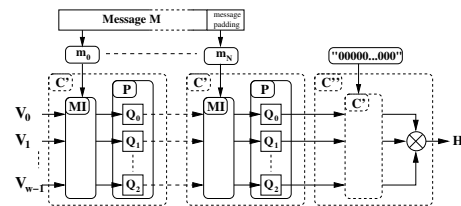


Fig. 9. Luffa

The round function consists of a message injection function (MI) and a permutation function (P). The MI can be implemented simply using an array of XOR gates as defined in

the specifications. The round function is more complex and consists of w non-linear permutation functions, Q_j , which execute 8 iterations of a step function. w is defined as 3 for Luffa-224/256, 4 for Luffa-384 and 5 for Luffa-512. Each Q_j performs an input tweak function followed by 8 iterations of the step function. The step consists of an S-box transformation, implemented in distributed ROM. The MixWord function is a linear permutation of two words and is implemented by a series of shifts and XORs. The final stage of the step function is AddConstant in which a predefined step constant is XORed to a single word of the input. The step constant is dependant on the current iteration of the round function.

The core unit in the implementation of Luffa is the Step function which can be executed in a single clock cycle while still maintaining a minimum clock delay. Due to the S-box, a clock delay is incurred here. To reduce any further clock delays, the output of one iteration of the step function is passed directly to the input of the next. In this way, one round of Q_j will take 8 clock cycles. Each Q_j can be executed in series or in parallel in order to target area or speed optimizations. The only differences between each instance are the step constants. Therefore, in order to implement each Q_j using a single instance of Q , a mux is required to control the selection of the constants. The implementation results given in this paper are based on fully parallel implementations.

I. SHAvite-3 - Biham et al.

The compression function for SHAvite-3 is a keyed permutation that is used with the Davies-Meyer construction [12]. All SHAvite-3 hash functions use the following steps to compress a message.

- Initialisation: $\#bits$ and h_0 are set to their initial values.
- Message Padding: message is padded into N 512-bit blocks.
- Message Compression: message is compressed by computing h_i , where i is the number of 512-bit blocks in the message. In parallel the key-expansion module accepts a 832-bit "key" to generate 144 32-bit words. The computing process of h_i can be expressed as: $h_i = CF_{256}^{S3}(h_{i-1}, M, salt, \#bits)$, in the case of 256-bit hash value, where M is a message, $SALT$ is a sub-key.
- Finalisation and Output: after compression function has processed all message blocks, the hash output is the 224/256 left-most bits of h_i

For CF_{512}^{S3} , the sizes of all the inputs are doubled.

To achieve a high throughput rate, a fast AES module is needed. Since the AES modules are processed sequentially, only one AES block is required in the compression function module. The architecture is shown in Fig. 10. For a parallel implementation, a second AES block is required for computation.

In the compression function architecture, the 256-bit chaining variable is split into two 128-bit parts, namely $state0$ and $state1$. the AES function using a 128-bit data bus processes $state1$ in one clock cycle. The output of the AES function will be fed back to be processed again. After three AES

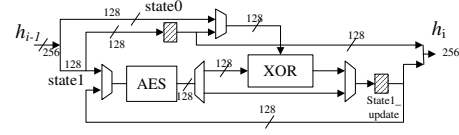


Fig. 10. SHAvite-3 high performance compression function architecture

computations, the output is XORed with the data from the $state0$ registers. after the XOR operation $state0$ and $state1$ update and input into the next round computation. In total, 12 rounds are required in CF_{256}^{S3} . The updated chaining variable or hash output is available after 37 clock cycles. The AES S-boxes are implemented using distributed ROM memory.

In key-expansion, in the non-linear stage 1 clock cycle is required to generate every four keys. In the linear stage 1 clock cycle to generate every eight keys. The total number of clock cycles required for generating all keys is 25. In order to produce a corresponding key ahead of the compression function, the key-expansion module needs to be triggered one clock cycle earlier than the compression function.

J. SIMD - Leurent et al.

SIMD is an iterated hash function, based on the Merkle-Damgard design, with a compression function based on a modified Davies-Meyer function using a Fiestel-like block cipher. The design of the the compression function using parallel Fiestel ladders allows for high throughput implementations on hardware.

There are two SIMD variants, SIMD-256 and SIMD-512. For any SIMD- n , $n \leq 256$, the message digest is a truncation of SIMD-256. Similarly, where $256 < n \leq 512$, the message digest is a truncation of SIMD-512. The inner state, S , is represented as a 4x4 matrix of 32-bit words for SIMD-256, or an 8x4 matrix for SIMD-512. SIMD- n uses the following operations to compute the hash of a message.

- Initialisation. An initial hash value H_{-1} is set to the corresponding IV. This is followed by padding.
- Message Compression. Each message block M_i is combined with the previous hash value H_{i-1} . Simultaneously, the message expansion of M_i is computed. A permutation of E is performed with both of these values as input. This consists of 4 rounds of fiestel ladders and is followed by a final round P . This process is illustrated in Fig. 11.
- Final Compression and Truncation. A final compression, C' is performed with H_k and $|M|$ as inputs, using a slightly modified message expansion.

The function E , shown in Fig. 11, consists of four rounds of parallel Fiestel Ladders, each composed of eight Fiestel Steps. The Fiestel Step is the core unit of the compression function. There are eight Fiestel Steps in each Ladder, with four Ladders in parallel in each Round (eight Ladders for SIMD-512). This structure is illustrated in Fig. 12. Registers are placed on the outputs of each Fiestel Step, as well as on each round, in order to minimize the critical path and therefore maximise the

clock frequency. Removing some of these registers, reduces the number of clock cycles required to complete an iteration of the compression function but the effect on the throughput is cancelled out by the reduction of the overall clock frequency.

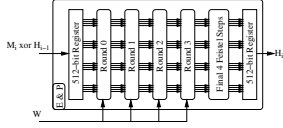


Fig. 11. SIMD Step

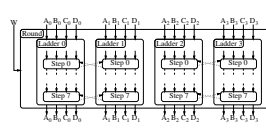


Fig. 12. SIMD Round

There are a total of four rounds in each compression function, followed by a final *half*-round consisting of four parallel Fiestel Ladders consisting of four Fiestel Steps each. The output of this final round is H^i , the hash output.

K. Skein 512 - Ferguson et al

Skein-512 is the primary proposal of the Skein family of algorithms. Skein uses the following steps to compress a message:

- A Unique Block Iteration (UBI) chaining mode takes in the chain value, the message and a 'Tweak' defined by an 128-bit configuration string derived from the message counter and UBI constants, which is in turn dependant on a 96-bit value of the message counter.
- The Threefish algorithm has 72 rounds consisting of four sets of four MIX functions followed by a permutation of the eight 64-bit words
- Each MIX function consists of a single addition, a rotation by a constant, and an XOR. The rotation constants repeat every eight rounds
- The key schedule generates the subkeys from the chain and a tweak, an 128-bit value derived from the message counter and constants
- A finalisation UBI stage consisting of a null message, a Tweak and the previous chain

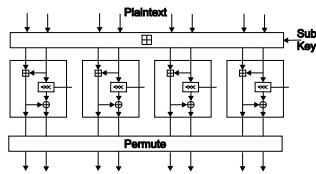


Fig. 13. Skein Round

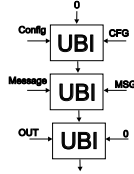


Fig. 14. UBI Hashing Mode

For our design of Skein-512 we unrolled four rounds of threefish, Fig. 13. In this way a UBI message block of Skein, shown in Fig. 14, takes 18 clocks for the rounds to complete, plus 5 for preprocessing and data loading. We use the precomputed IV to remove the configuration stage and thus remove the initial message block from the latency. Each subsequent message block and the output block are calculated in the same way. The tweak, which ensures each message block is different, is generated by the counter in the padding. Again, the Adders and XORs were generated in a generic fashion and the

rotation operations were implemented through simple wiring, with multiplexers to select the particular subround rotation.

L. Other Designs

Here we give a brief description of any updates or modifications from our previous work in [4].

a) *CubeHash - Bernstein*: For the second round of the competition, the Cubehash specification variants were updated. The designs recommended for round 1 were CubeHash8/1-*h*. For round 2, the suggested designs are of the form CubeHash8/32-*h*. This has the effect of giving a 16x speedup to the designs.

b) *Shabal - Saphir research project*: There are no changes to Shabal. The design used was our parallel adder version (#19). Similar to Skein, we use the IVs to remove the first two rounds.

c) *Grøstl - Gauravaram et al.*: There are also no changes to Grøstl. The design used was our BRAM parallel version (#5, #11), with BRAM replaced by distributed ROM.

IV. RESULTS

Table III gives the clock count for the various designs. As can be seen from the table, some hash designs require extra time to load in the padding scheme, while others have finalisation stages comprising a number of rounds. For calculating the throughput, as the size of the message to be hashed increases, these padding and finalisation stages will have less of an impact on the overall calculation time. However for short messages, they have a big impact. We therefore define a short message as the time required to process the padding, a message block and finalisation, and a long message as just the time to process the message block. Note that each hash function operates over the state size given in Table I, and so designs with smaller state sizes will require a larger number of rounds to hash the same amount of data as a design with a large state size. This is also reflected in the throughput.

The larger state sizes however are affected by the loading latency as explained in Section II. Where the time required to hash the message is larger than the time required to load the message this only affects the initial message loading, but in cases where the load time is longer than the hash time (denoted * in Tables III and IV), there will be a wait delay as the hash waits for data. For these cases the clock count for the throughput needs to take this additional delay into consideration. Not given here is the output message load time, which in all cases is the hash digest size/output bus size.

The timing and area results for the implemented hash functions for the Xilinx Virtex-5 *xc5v1x330T-2-ff1738* are given in Table IV. The *-w* designation, defines the results inclusive of the wrapper, while *-nw* gives the hash function as a stand alone entity. Included also, are the updated results from [4] for Section III-L. The throughput results given are inclusive of the wrapper with *TP-s* using Table III's clock count for a short message, and *TP-l* using the clock count for a long message where the padding and finalisation stages will have little impact on the message. Fig. 15 and 16 give

TABLE III
HASH FUNCTION TIMING RESULTS

Hash Design	32-bit load	Extra Padding	Padding #Cycles	Message Rounds	Round #Cycles	Long Msg #Cycles	Final Rounds	Final #Cycles	Short Msg #Cycles
SHA224/256	16	0	0	64	1	65	0	0	65
SHA384/512	32	0	0	80	1	81	0	0	81
Blake224/256	16	0	0	10	4	40	0	0	40
Blake384/512	32	0	0	14	4	56	0	0	56
BMW224/256*	16	0	0	1	4	4	1	3	7
BMW384/512*	32	0	0	1	4	4	1	3	7
Cubehash	8	0	0	16	17	17	160	161	178
Echo224/256*	48	0	0	8	1	8	1	1	9
Echo384/512*	32	0	0	10	1	10	1	1	11
Fugue224/256	1	2	1	1	7	7	13	91	98
Fugue384	1	2	1	1	10	10	20	180	190
Fugue512	1	2	1	1	13	13	22	264	277
Grøstl224/256*	16	0	0	10	1	10	0	0	10
Grøstl384/512*	32	0	0	14	1	14	0	0	14
Hamsi224/256	1	3	1	3	2	6	6	24	31
Hamsi384/512	2	3	1	6	2	12	12	48	61
JH	16	1	1	35	1	38	0	0	38
Keccak224*	36	0	0	24	1	25	0	0	25
Keccak256*	34	0	0	24	1	25	0	0	25
Keccak384*	26	0	0	24	1	25	0	0	25
Keccak512	18	0	0	24	1	25	0	0	25
Luffa224/256*	16	0	0	8	1	8	1	8	16
Luffa384*	16	0	0	8	1	8	2	16	24
Luffa512*	16	0	0	8	1	8	2	16	24
Shabal	16	0	0	1	30	30	3	150	200
SHAvite3-224/256	16	0	0	12	3	36	1	1	37
SHAvite3-384/512	32	0	0	14	4	56 (70)	1	1	71
Simd224/256	16	1	1	4	8	32(41)	0.5	4	36(45)
Simd384/512	32	1	1	4	8	32(41)	0.5	4	36(45)
Skein	16	0	0	18	22	396	18	396	792

TABLE IV
HASH FUNCTION IMPLEMENTATION RESULTS

#	Hash Design	Area-sw (slices)	Max.Freq-w (MHz)	Area-sw (slices)	Max.Freq-w (MHz)	TP-1 (Mbps)	TP-1/Area (Mbps/slice)	TP-s (Mbps)	TP-s/Area (Mbps/slice)
1	SHA-2-256	1,019	125.063	656	125.125	985	0.966	985	0.966
2	SHA-2-512	1,771	100.04	1,213	110.096	1,264	0.713	1,264	0.713
3	BLAKE-32	1,653	91.349	1,118	118.064	1,169	0.707	1,169	0.707
4	BLAKE-64	2,888	71.048	1,718	90.909	1,299	0.449	1,299	0.449
5	BMW-256*	5,584	14.308	4,997	14.016	457	0.081	457	0.081
6	BMW-512*	9,902	8.985	9,810	10.004	287	0.028	287	0.028
7	Cubehash	1,025	166.667	695	166.833	2509	2.447	239	0.233
8	ECHO-256*	8,798	161.212	7,372	198.926	5373	0.61	5373	0.61
9	ECHO-512*	9,130	166.667	8,633	166.694	18133	1.986	5666	0.608
10	Fugue-256	2,046	200	1,689	200.04	914	0.446	60	0.029
11	Fugue-384	2,622	200.08	2,380	200.08	640	0.244	33	0.012
12	Fugue-512	3,137	195.81	2,596	200.16	481	0.153	22	0.007
13	Grøstl-256*	2,579	78.064	2,391	101.317	3242	1.257	3242	1.257
14	Grøstl-512*	4,525	113.122	4,845	123.396	3619	0.799	3619	0.799
15	Hamsi-256	1,664	67.195	1,518	72.411	358	0.215	69	0.041
16	Hamsi-512	7,364	14.931	6,229	16.51	79	0.01	15	0.002
17	JH	1,763	144.113	1,291	250.125	1941	1.1	1941	1.1
18	Keccak-224*	1,971	195.733	1,117	189	5915	3	5915	3
19	Keccak-256*	1,971	195.733	1,117	189	6263	3.17	6263	3.17
20	Keccak-384*	1,971	195.733	1,117	189	8190	4.15	8190	4.15
21	Keccak-512	1,971	195.733	1,117	189	8518	4.32	8518	4.32
22	Luffa-256*	2,796	166.667	2,221	166.667	2666	0.953	2666	0.953
23	Luffa-384*	4,233	166.75	3,740	166.75	2668	0.63	1778	0.42
24	Luffa-512*	4,593	166.667	3,700	166.75	2666	0.58	1777	0.58
25	Shabal	2,512	143.472	1,583	148.038	1469	0.584	367	0.146
26	SHAvite3-256	3,776	82.277	3,125	109.17	1170	0.309	1138	0.301
27	SHAvite3-512	11,443	63.666	9,775	59.4	931	0.081	918	0.08
28	SIMD-256	24,536	107.2	22,704	107.2	1338	0.054	1338	0.054
29	SIMD-512	44,673	107.2	43,729	107.2	2677	0.059	2677	0.059
30	Skein-512	3,027	83.577	1,938	83.654	108	0.035	54	0.018

a graphical representation for long message TP/area, while the short message TP/area are given in Fig. 17 and 18. We highlight SHA-2 (#1, #2) as the benchmark to compare the others against.

From the tables and graphs, it can be seen that for our baseline FPGA implementations, three designs (Keccak, Grøstl and JH) consistently outperformed SHA-2 in all above categories. Five other designs (ECHO, Luffa, Cubehash, SIMD and BLAKE) outperformed SHA-2 at least half of the time.

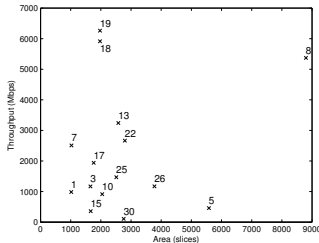


Fig. 15. 224/256 Long Mes

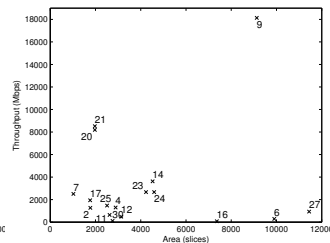


Fig. 16. 384/512 Long Mes

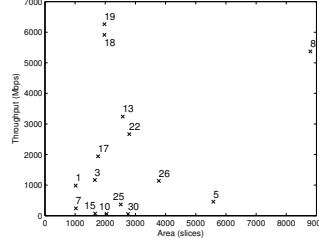


Fig. 17. 224/256 Short Mes

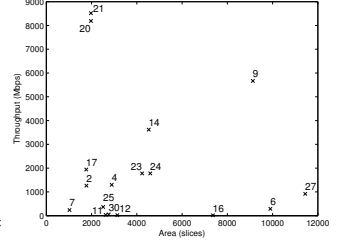


Fig. 18. 384/512 Short Mes

V. CONCLUSIONS

In this paper we presented what we believe to be a methodology for fair and accurate comparisons of the SHA-3 hash functions. We implemented and tested as many designs as was necessary to obtain full coverage of all of the hash variants as required by NIST. We developed a hardware wrapper to allow inclusion of padding and interfacing, to obtain the full timing and area analysis, and for completeness compared the area and speed of our hash designs both internal and external of this wrapper. Finally we presented throughput results for both long and short hash messages inclusive of this wrapper.

REFERENCES

- [1] NIST, "National institute of standards and technology. [docket no.: 070911510751201] announcing request for candidate algorithm nominations for a new cryptographic hash algorithm (SHA3) family," Federal Register, November 2007.
- [2] K. Kobayashi, J. Ikegami, S. Matsuo, K. Sakiyama, and K. Ohta, "Evaluation of hardware performance for the SHA-3 candidates using SASEBO-GII," Cryptology ePrint Archive, Report 2010/010, 2010.
- [3] S. Tillich, M. Feldhofer, M. Kirschbaum, T. Plos, J.-M. Schmidt, and A. Szekely, "High-speed hardware implementations of BLAKE, Blue Midnight Wish, FugueHash, ECHO, Fugue, Grøstl, Hamsi, JH, Keccak, Luffa, Shabal, SHAvite-3, SIMD, and Skein," Cryptology ePrint Archive, Report 2009/510, 2009.
- [4] B. Baldwin, A. Byrne, M. Hamilton, N. Hanley, R. P. McEvoy, W. Pan, and W. P. Marnane, "FPGA implementations of SHA-3 candidates: CubeHash, Grøstl, LANE, Shabal and Spectral Hash," in *Digital Systems Design, Euromicro Symposium on*, 2009, pp. 783–790.
- [5] B. Baldwin, A. Byrne, L. Lu, M. Hamilton, N. Hanley, M. O'Neill, and W. P. Marnane, "A hardware wrapper for the SHA-3 hash algorithms," in *21st Irish Signals and Systems Conference, ISSC*, 2010.
- [6] E. Biham and O. Dunkelman, "A framework for iterative hash functions - HAIFA," Cryptology ePrint Archive, Report 2007/278, 2007.
- [7] R. C. Merkle, "One way hash functions and DES," in *Advances in Cryptology — CRYPTO '89*, ser. LNCS, G. Brassard, Ed., vol. 435. Springer-Verlag, 1989, pp. 428–446.
- [8] I. Damgård, "A design principle for hash functions," in *Advances in Cryptology — CRYPTO '89*, ser. LNCS, G. Brassard, Ed., vol. 435. Springer-Verlag, 1989, pp. 416–427.
- [9] NIST. (2009, September) Second round candidates. [Online]. Available: http://csrc.nist.gov/groups/ST/hash/sha-3/Round2/submissions_rnd2.html
- [10] E. Biham, R. J. Anderson, and L. R. Knudsen, "Serpent: A new block cipher proposal," in *Fast Software Encryption — FSE '98*, ser. LNCS, S. Vaudenay, Ed., vol. 1372. Springer-Verlag, 1998, pp. 222–238.
- [11] G. Bertoni, J. Daemen, M. Peeters, and G. V. Assche, "On the indifferentiability of the Sponge Construction," pp. 181–197, 2008.
- [12] R. S. Winternitz, "A secure one-way hash function built from DES," in *IEEE Symposium on Security and Privacy*. IEEE Press, 1984, pp. 88–90.